now give me the steps to build the modules and test them to verify their functionality and lock them.

[AURA-STRICT-PROTOCOL] is actively enforcing the "Lock & Move" methodology.

To build an institutional-grade system, we must construct the architecture from the bottom up. No higher-level module can be built until its foundational dependencies pass atomic tests and are cryptographically or physically "locked."

Here is the exact step-by-step roadmap to build, test, verify, and lock the core modules of AURA.

THE "LOCK & MOVE" MECHANISM (Standard Operating Procedure)

For every module below, the enforcement protocol follows this exact sequence:

1. **Build:** Write the code in the designated versioned folder (e.g., `v1.0.0`).
2. **Test:** Execute atomic unit/integration tests (`pytest` for Python, MT5 Strategy Tester for MQL5).
3. **Verify:** Confirm output metrics (latency, memory, mathematical accuracy) meet strict thresholds.
4. **Lock:** Set file permissions to Read-Only (`attrib +R` on Windows, `chmod 444` on Unix) and log the module's SHA-256 hash in a central `aura_registry.json`. To edit later, the MCP agent must duplicate the folder to `v1.0.1` and build there.

PHASE 1: The Data & Nervous System

These modules form the baseline for all subsequent RL and UI tasks.

Module 1: High-Speed Data Pipeline (Polars)

- **Build:** Create `data/polars_converter.py`. Write a script that scans `data/csv_landing`, reads M1/M15/H1 CSVs, standardizes the OHLCV columns, handles missing data (forward fill), and writes to `data/parquet_capsules`.
- **Test:** Place a 10-year 1-Minute CSV (e.g., 5GB) in the landing folder. Run the script.
- **Verify:**
  - Conversion must complete in under 10 seconds.

    - Output `.parquet` file size must be >70% smaller than the CSV.
    - Data types must be strictly typed (e.g., `Float32` for prices to save RAM, `Datetime64` for time).
- **Lock Action:** Lock `polars_converter.py`.

Module 2: Local ZMQ Bridge (MT5 ↔ Python)

- **Build:**
    - Python: `backend/core/zmq_bridge.py` using `pyzmq`. Set up a `SUB` socket to listen for tick data and a `REP` socket to receive order execution requests.
    - MQL5: `ea/Include/ZmqConnector.mqh`. Set up a `PUB` socket to broadcast ticks and a `REQ` socket to send order results.
- **Test:** The "Ping-Pong" Latency Test. Send 10,000 dummy ticks from MT5 to Python, and have Python echo back a "received" confirmation.
- **Verify:** Average round-trip latency must be < **2 milliseconds** on localhost. Zero dropped packets.
- **Lock Action:** Lock `zmq_bridge.py` and `ZmqConnector.mqh`.

PHASE 2: The Brain & Environment

Building the reality in which the RL Agent will live and learn.

Module 3: The Harsh Simulated Gym (RL Environment)

- **Build:** Create `rl/environments/HarshSimGym.py` subclassing `gymnasium.Env`. Implement `step()` and `reset()`. Integrate the Parquet capsule loader. Add the **Spread/Commission Penalty Engine** and simulated slippage based on volume.
- **Test:** The "Random Gambler Test." Instantiate a dummy agent that buys/sells randomly at every time step.
- **Verify:** The agent's equity curve must consistently hit 0 (ruin) quickly due to the realistic spread and commission bleed. If the agent survives by random chance, the environment is too forgiving and must be fixed.
- **Lock Action:** Lock `HarshSimGym.py`.

Module 4: The Sixth Sense (Alternative Data Agent via Ollama)

- **Build:** Create `agents/sentiment_ollama_agent.py`. Connect to the local Ollama API (e.g., using `llama3` or `mistral`). Write a specific prompt template: *"Classify this financial news on a scale of -1.0 (bearish) to 1.0 (bullish)."*
- **Test:** Feed an array of 50 historically known news headlines (e.g., "NFP massively beats expectations", "Fed unexpectedly hikes rates").
- **Verify:**

    - Score accuracy matches consensus > 90%.
    - Inference time per headline is < 200ms (to ensure it doesn't bottleneck live trading).
- **Lock Action:** Lock `sentiment_ollama_agent.py`.

PHASE 3: Execution & Control

Building the enforcer that pulls the trigger.

Module 5: MT5 Master EA & Risk Management Agent

- **Build:**
    - Create `ea/Experts/EURUSD_Master_v1/MasterEA.mq5`.
    - Implement the Risk Hierarchy: Global Symbol limits first, then Strategy-specific logic. Include trailing SL/TP algorithms.
    - Build the JSON parser to read the *Institutional Parameter Skeleton*.
- **Test:** MT5 Visual Strategy Tester. Send predefined ZMQ commands from Python: `BUY 0.1 EURUSD SL:20 TP:60`.
- **Verify:**
    - Trade executes flawlessly in the tester.
    - Trailing SL activates exactly at the designated pip threshold.
    - If a trade breaches the "Max Daily Drawdown %", the Master EA immediately halts all trading for the symbol.
- **Lock Action:** Compile `.ex5`, verify execution, and Lock `MasterEA.mq5` and `RiskManager.mqh`.

Module 6: FastAPI Core & Next.js Frontend (The Dashboard)

- **Build:**
    - FastAPI: REST endpoints for initiating Backtests, polling Optuna results, and WebSockets for live equity streaming.
    - Next.js: Build the UI components (Symbol Dropdown, Backtest Gym Button, Paper Trading Dashboard). Connect TanStack Query to FastAPI.
- **Test:** E2E (End-to-End) Pipeline Test. Click "Convert to Parquet" on the UI -> Watch FastAPI trigger Polars -> Watch UI progress bar update via WebSocket.
- **Verify:** UI state reflects backend state accurately without manual page refreshes.
- **Lock Action:** Lock core API routers and UI state management hooks.

PHASE 4: Strict Pipeline Progression Integration

Once all 6 modules are Locked, you execute the **[AURA-STRICT-PROTOCOL] Progression Test:**

1. **Backtesting Trigger:** Use the UI to run the RL Agent in the `HarshSimGym`.
2. **0-Trade Auto-Tune:** Intentionally set terrible parameters. Verify the *Parameter Detection Agent* intercepts the 0-trade result and auto-adjusts before Optuna takes over.
3. **Paper Trading Promotion:** If the backtest yields a Sharpe > 1.5, verify the *Pipeline & Version Control Agent* automatically provisions the model into the **Dress Rehearsal (Live ZMQ with Fake Money)** environment.
4. **Live Lock:** Only after 30 days of simulated live ticks (can be accelerated in tests) with positive metrics will the system unlock the "Deploy Live Capital" API endpoint.